

26. Reading and writing binary files out of order

Herman Kamper

So far, every file we have read or written was processed from beginning to end. We opened it, started at the first byte and moved through it in order, one line or one block of bytes at a time. That works well when you want to process everything. But many applications are not like that.

Suppose a file holds the first five batsmen of the Protea cricket team and the runs they made:

Bavuma	128
Ackerman	18
Bedingham	17
DeZorzi	51
Markram	34

If all you want are the runs of the third batsman, it is silly to read the first two entries just to get past them. With a file of a million entries, it would be hopeless. What you want is to jump straight to the third entry, read it, and carry on from there.

In this lecture we see how. We will move around inside a file, which is called random access, and we will do it with binary files, where it is easiest. The only new function is `fseek`. Along the way we will finally say what the mysterious `FILE *` really points at.

Random-access files

Binary files are a reminder of what we saw in the previous lecture: you have some variables in memory, and you copy their bytes directly to the file with `fwrite`, with no conversion to text. You can copy them back into variables with `fread`. (For text files the equivalents are `fprintf` and `fscanf` or `fgets`. For binary files they are `fwrite` and `fread`.)

Binary files make it easy to move around in a file, because you know exactly how many bytes every record takes up. Suppose we have a structure of 100 bytes in total, and we write four of them to a file, one after the other. The file is then 400 bytes long, and each record sits at a position that is easy to calculate:

byte offset:	0	100	200	300	400
	-----	-----	-----	-----	
	record 0	record 1	record 2	record 3	
	100 bytes	100 bytes	100 bytes	100 bytes	
	-----	-----	-----	-----	

Record 0 starts at byte 0, record 1 at byte 100, record 2 at byte 200, and so on. In general, if every record takes S bytes, record number n (counting from zero) starts at byte $n \times S$. So far, we have always started at the start of the file, read one record, then the next, and the next. What we want now is to move straight to any record and read or write from there.

A file that we use in this way, jumping to specific places and reading or writing from them, is called a random-access file. The key requirement is that the records have a fixed length, so that the position of any record can be calculated. This is the reason text files are normally processed sequentially: their lines have different lengths, so you cannot work out where line 500 starts without reading the 499 lines before it. A binary file of structures has no such problem.

What a FILE * really is

To understand `fseek` we need to look more carefully at something we have been using since the start of the files lectures. We kept calling `fPtr` a “handle” for a file. Now we can say what it is.

Consider these two lines:

```
FILE *fPtr;  
fPtr = fopen("file.bin", "rb");
```

The first line declares a pointer, exactly like `int *p` would. The box `fPtr` holds an address; on a 64-bit machine, that box is eight bytes. But what does it point at? The type is `FILE`, and `FILE` is a structure.

When `fopen` is called, it creates one of these `FILE` structures somewhere in the computer’s memory, and it returns the address of that structure. The assignment then puts that address in `fPtr`, so `fPtr` points at the structure. What is inside it is up to the implementation of the library, and we do not need to know most of it. But one member is important to us. It is called the file position pointer (the C standard calls it the file position indicator), and it records where in the file the next read or write will take place.

This member is an unusual kind of pointer. It does not point at another address in the computer’s memory. It points at a byte in a file on the hard drive. When you open a file with `fopen`, the file position pointer points at the first byte of the file. The figure shows all of this at once, with the computer’s memory on the left and the hard drive on the right:

Now we can describe exactly what happens with the reading and writing functions we know. When you do an `fread` of two bytes, the two bytes are copied into memory, and the file position pointer moves forward by two bytes. If you then read four bytes, it moves forward by another four. That is all “moving through the file” means. In the same way, each `fwrite` puts bytes at the position the pointer points to and moves it forward by the number of bytes written.

You can safely go on thinking of `fPtr` as a handle that points at the file on the disk. Under the hood, there is this structure in between, and its file position pointer is the thing that moves.

```
FILE *fPtr;
fPtr = fopen("file.bin", "rb");
```

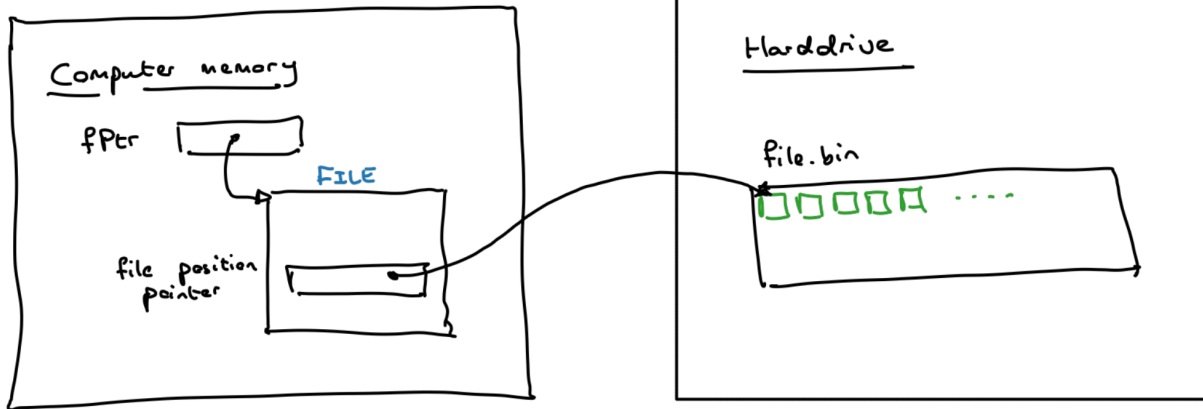


Figure 1: What `fopen` creates. In memory, `fPtr` points at a `FILE` structure. Inside that structure is the file position pointer, which points at a byte of `file.bin` on the hard drive. Right after `fopen` it points at the first byte.

The `fseek` function

The new function for this lecture lets you set the file position pointer to wherever you like. After that, the next read or write happens there. Here is its prototype:

```
int fseek(FILE *stream, long int offset, int whence);
```

It takes three arguments:

- `stream` is the file handle, the `FILE *` that you got from `fopen`.
- `offset` is how far to move, in bytes.
- `whence` says what the offset is measured from.

The last argument has three possible values. They are written as names, which are defined in `stdio.h`:

Value	The offset is measured from
<code>SEEK_SET</code>	the start of the file
<code>SEEK_CUR</code>	the current position of the file position pointer
<code>SEEK_END</code>	the end of the file

The offset can be negative. Combined with `SEEK_END`, that lets you point at the very end of the file and then step back, say, four bytes. With `SEEK_CUR`, you can move backwards from wherever you currently are. These two are less common than the first one. In practice, nearly all the time you open a file, jump to a position counted from the start with `SEEK_SET`, and read or write from there.

The three names are really just integers. If you print them, you will see 0, 1 and 2 on most systems, but you should always use the names and never the numbers.

The function returns 0 if the move worked, and a non-zero value if it did not.

Here is an example. Suppose a file holds a series of `struct coord` records, each with an `x` and a `y`:

```
struct coord
{
    double x;
    double y;
};
```

To write a point as the fifth record in the file, we have to skip the first four records. They occupy `4 * sizeof(struct coord)` bytes, so that is the offset from the start:

```
struct coord xPnt = {1.0, 5.0};

fseek(fPtr, 4 * sizeof(struct coord), SEEK_SET);
fwrite(&xPnt, sizeof(struct coord), 1, fPtr);
```

The `fseek` moves the file position pointer past four records, and the `fwrite` writes one record there, replacing whatever was in the fifth record before. The records before it are not touched. Note the numbering: skipping four records puts us at record index 4, which is the fifth record, because we count from zero.

Reading works in exactly the same way. First move, then read:

```
struct coord xPnt;

fseek(fPtr, 4 * sizeof(struct coord), SEEK_SET);
fread(&xPnt, sizeof(struct coord), 1, fPtr);
```

This reads the fifth record into `xPnt`. To read several records at once, you give `fread` a pointer to an array and the number of elements as the third argument, exactly as in the previous lecture.

So the rule for the position of record number i (counting from zero) is:

```
i * sizeof(struct coord)
bytes from the start of the file.
```

A bigger example: a bank

Let us put this to work. Imagine we are writing the software for a bank (one that makes a lot of money). Each customer has an account number, a last name, a first name and a balance, which we put in a structure:

```
struct clientData
{
    int acctNum;           // account number
    char lastName[15];     // last name
    char firstName[10];    // first name
    double balance;        // account balance
};
```

The bank has 100 customers, and we want one file that holds the records of all of them, so that we can update the balance of any one customer without touching the others. We will write three programs in turn. The first creates a file with 100 blank records. The second uses `fseek` to update one customer. The third reads customers back.

Before we start, one question about the size of a record. The members take 4 bytes for the integer, 15 for the last name, 10 for the first name and 8 for the double, which is 37 bytes in all. If you ask C, however, for `sizeof(struct clientData)`, the answer on most machines is 40. The reason is that the compiler is allowed to insert unused padding bytes between members, because many machines want a double to start at an address that is a multiple of 8. Here, the three bytes after the first name are padding, which brings the double to byte 32. This is why we always use `sizeof` rather than adding up the sizes of the members by hand. The 40 bytes are the truth about what lies in memory, and what `fwrite` copies.

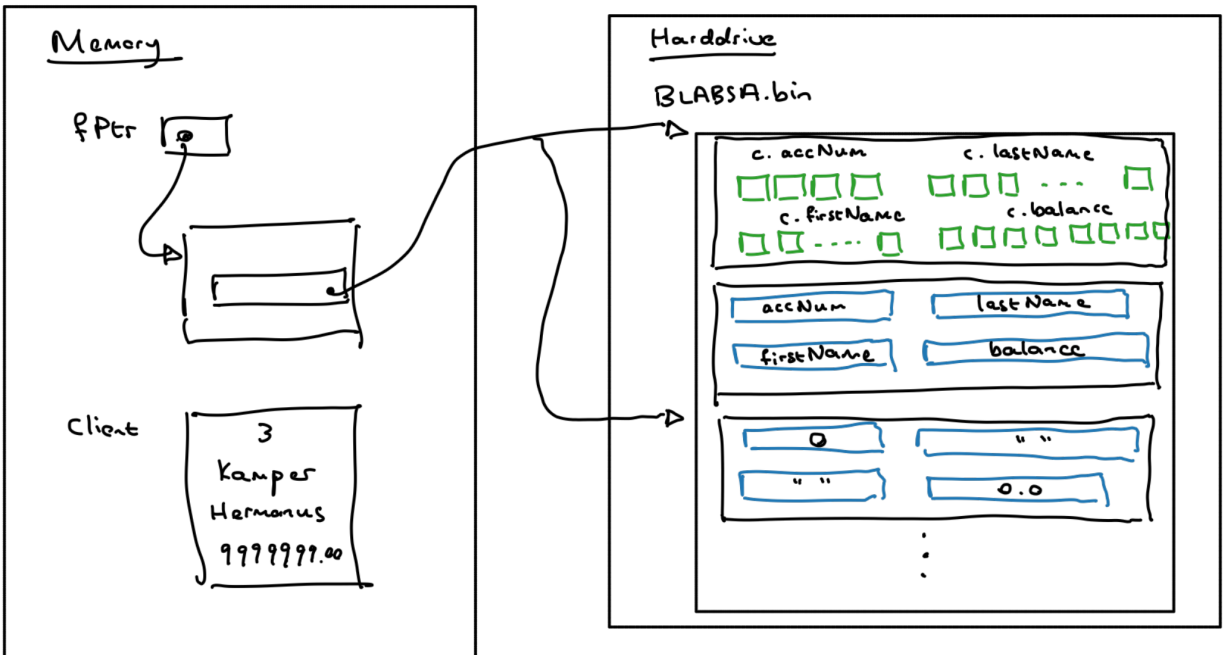


Figure 2: A client record in memory, and the same record in the file on the hard drive. The file BLABSA.bin is a long row of equally large records. Each one holds an account number, a last name, a first name and a balance. The blank records contain 0, empty strings and 0.0. The record for customer 3 is copied over the third blank record.

Program 1: a file of blank records

The first program creates the file and fills it with 100 blank records. A blank record has account number 0, empty names and a balance of 0.0. We open the file with "wb", which creates it, and write the same blank structure 100 times:

```
#include <stdio.h>

struct clientData
{
    int acctNum;           // account number
    char lastName[15];     // last name
    char firstName[10];    // first name
    double balance;        // account balance
};

int main()
{
    int i;
    struct clientData blank = {0, "", "", 0.0};
    FILE *fPtr;

    fPtr = fopen("BLABSA.bin", "wb");

    if (fPtr == NULL)
    {
        printf("ERROR - file could not be opened!\n");
    }
    else
    {
        for (i = 0; i < 100; i++)
        {
            fwrite(&blank, sizeof(struct clientData), 1, fPtr);
        }
        fclose(fPtr);
        printf("Wrote 100 blank clients\n");
    }

    return 0;
}
```

The `fwrite` copies the address of `blank` for `sizeof(struct clientData)` bytes, once, to the file. The file position pointer moves forward by 40 bytes each time, so the second write goes straight after the first, and so on.

After running it, the file is 4000 bytes long, which is 100 records of 40 bytes each, and if you open it in a text editor you see only noise.

One thought about what is in that file is worth stopping at. You might ask whether each record occupies one line of the file. The answer is that a line does not exist. The only things in a file are bytes. The 4000 bytes lie one after another on the disk, and nothing in them marks a boundary between one record and the next. We know where record 3 starts only because we know that every record is 40 bytes long. Even a text file has no lines as such: it only has a byte that represents “new line”, and the text editor starts a new line on the screen when it sees one.

Program 2: updating one customer

The second program asks for an account number, a last name, a first name and a balance, and writes the new details to the right place in the file, without touching any of the other records. It repeats this until the user types account number 0.

There is one question to settle before the code: which mode do we use to open the file? Not "wb": it throws away the existing contents, which would destroy all the records. Not "rb" either, since we need to write. We want to keep what is there and be able to change parts of it, so we use "rb+", which opens an existing file for reading and writing without erasing it:

```
#include <stdio.h>

struct clientData
{
    int acctNum;
    char lastName[15];
    char firstName[10];
    double balance;
};

int main()
{
    struct clientData client = {0, "", "", 0.0};
    FILE *fPtr = fopen("BLABSA.bin", "rb+");

    if (fPtr == NULL)
    {
        printf("ERROR - file could not be opened!\n");
    }
    else
    {
        printf("Enter account number (1 to 100, 0 to end): ");
        scanf("%d", &client.acctNum);

        while (client.acctNum != 0)
        {
            printf("Enter last name, first name, balance: ");
            scanf("%14s%9s%lf", client.lastName, client.firstName,
```

```

        &client.balance);

    fseek(fPtr, sizeof(struct clientData) * (client.acctNum - 1),
          SEEK_SET);
    fwrite(&client, sizeof(struct clientData), 1, fPtr);

    printf("Enter account number (1 to 100, 0 to end): ");
    scanf("%d", &client.acctNum);
}

fclose(fPtr);
}

return 0;
}

```

The first `scanf` has nothing to do with files: it reads the account number into the structure, and the loop runs as long as the number is not 0. The `%14s` and `%9s` leave room for the string-terminating `\0` in the two name arrays, so that a long name cannot overflow them. A single `scanf` can read several values at once, as here.

The important lines are the `fseek` and the `fwrite`. Suppose the user enters account 3. We want to write to the record of account 3, and that is the third record in the file. To get to the third record we need to skip two records, which is $3 - 1$. That is why the offset is `sizeof(struct clientData) * (client.acctNum - 1)`: account numbers start at 1, but the record numbers in the file start at 0, so account number n is record index $n - 1$.

The `fseek` jumps to the start of record 3, and then `fwrite` copies the 40 bytes of `client` from memory straight over the third blank record, as the figure showed. The bytes are not placed “on a line”: they are just dumped there, over the 40 bytes that were there before. And `fwrite` knows how many bytes to write, and therefore where to stop, because we told it: `sizeof(struct clientData)`, once. Everything outside those 40 bytes is untouched.

Suppose we run it, entering account 3 with the details Kamper Herman 999999, account 1 with Naidoo Aisha 1250.75, and then 0 to stop. The file stays 4000 bytes long, with two of the 100 records changed.

Program 3: reading the customers

Now we need a way to check that it worked, and so a third program. It reads the file and prints every customer that has been captured, that is, every record whose account number is not 0. The file is opened for reading only:

```
#include <stdio.h>

struct clientData
{
    int acctNum;
    char lastName[15];
    char firstName[10];
    double balance;
};

int main()
{
    struct clientData client;
    FILE *fPtr = fopen("BLABSA.bin", "rb");

    while (fread(&client, sizeof(struct clientData), 1, fPtr) == 1)
    {
        if (client.acctNum != 0)
        {
            printf("%d: %s %s has R%.2f in their account\n",
                client.acctNum, client.firstName, client.lastName,
                client.balance);
        }
    }

    fclose(fPtr);

    return 0;
}
```

It prints:

```
1: Aisha Naidoo has R1250.75 in their account
3: Herman Kamper has R999999.00 in their account
```

Each `fread` reads one record from wherever the file position pointer currently is and moves it forward by 40 bytes. So the loop steps through the file record by record, without any `fseek` at all. The other 98 records are blank and are skipped by the `if`.

There is one detail in the loop condition. The function `fread` returns the number of items it managed to read. We asked for one record, so the return value is 1 when it worked, and 0 when it failed, for instance because we reached the end of the file. So the loop stops as soon as the

return value is not 1.

You will also see this loop written in the way we met for text files, with `feof`:

```
while (!feof(fPtr))
{
    result = fread(&client, sizeof(struct clientData), 1, fPtr);

    if (result != 0 && client.acctNum != 0)
    {
        /* print the customer */
    }
}
```

This one needs the `result != 0` test. As we saw earlier, `feof` only becomes true after a read has failed. So the last trip through the loop is the one where `fread` fails. In practice it leaves `client` as it was, still holding the last record that was read. If account 100 holds a real customer, then without the test on `result` that customer is printed twice. With the file above, where account 100 is blank, you would not notice, which is exactly what makes this bug so easy to miss. Testing the return value of `fread` directly, as in the first version, is simpler.

Finally, we do not have to read the whole file. To look up one account, we jump to its record and read it:

```
struct clientData client;
int accountNo;
FILE *fPtr = fopen("BLABSA.bin", "rb");

printf("Which account (1 to 100)? ");
scanf("%d", &accountNo);

fseek(fPtr, sizeof(struct clientData) * (accountNo - 1), SEEK_SET);
fread(&client, sizeof(struct clientData), 1, fPtr);
fclose(fPtr);

printf("%s %s has R%.2f in their account\n",
       client.firstName, client.lastName, client.balance);
```

For account 3, the `fseek` jumps directly to the third record, the `fread` copies it into `client`, and we print it. No other record is read. This is the whole point of random access: the time it takes to find a customer no longer depends on where in the file the customer is. For account 2, which has not been captured, the record is a blank one, so a real program would test whether `client.acctNum` is 0 and report that the account does not exist yet.

How big is the file?

A question that comes up is how C knows where the end of a file is. We do not need the answer to use `SEEK_END`, but it is useful to know how to find the size of a file ourselves. Move to the end of the file and ask where you are:

```
fseek(fPtr, 0, SEEK_END);  
long size = ftell(fPtr);
```

The function `ftell` returns the current position of the file position pointer, counted in bytes from the start of the file.¹ After seeking to the end, that is the size of the file in bytes. For our bank file it is 4000. The number of records in a file of fixed-size records is then:

```
size / (long) sizeof(struct clientData)
```

which is 100.

The same trick lets us reach the last record without knowing how many records there are. The offset is negative: we step back by one record's worth of bytes from the end. The cast to `long` is needed because `sizeof` gives an unsigned number, and a negative offset would otherwise turn into an enormous positive one:

```
fseek(fPtr, -(long) sizeof(struct clientData), SEEK_END);  
fread(&client, sizeof(struct clientData), 1, fPtr);
```

Note again that `feof` is a separate matter. It does not say how big the file is, and it does not tell you that you are *at* the end. It only tells you that a read has already gone past it.

Summary

Sequential access reads and writes a file from the start, one piece after another. Random access lets you jump to any position in a file and read or write from there. It is easiest with binary files of fixed-length records, where the position of record *i* is simply $i * \text{sizeof}(\text{record})$ bytes from the start.

- A `FILE *` points at a `FILE` structure created by `fopen`. Inside it is the file position pointer, which records where in the file the next read or write takes place. Every `fread` and `fwrite` moves it forward by the number of bytes transferred.
- `fseek(stream, offset, whence)` sets the file position pointer. The offset is in bytes and can be negative. `SEEK_SET` measures from the start of the file, `SEEK_CUR` from the current position and `SEEK_END` from the end. The one you will use most is `SEEK_SET`.
- To update or read record *n* (counting from zero), do `fseek(fPtr, n * sizeof(record), SEEK_SET)` and then `fwrite` or `fread` one record.
- Open a file for updating with `"rb+"`. Opening with `"wb"` destroys the existing contents.
- Always use `sizeof` for the size of a record: the compiler may insert padding, so the size can be more than the sum of the members.

¹For Computer Programming 143, you do not need to know what the `ftell` function does.

- `fread` returns the number of items read, so `fread(...) == 1` is a good loop condition for one-record reads. `ftell` gives the current position, and after seeking to the end, the size of the file.
- A file is only bytes. There are no lines in it, only bytes, some of which happen to represent a newline.

Exercises

1. Write a program that stores the five batsmen at the start of this lecture in a binary file, as records with a name of up to 19 characters and an integer score. Then write a second program that asks for a number from 1 to 5 and prints the score of that batsman, reading only that one record with `fseek` and `fread`.
2. Modify the bank program so that it checks the account number before using it, and refuses any number outside the range 1 to 100. What could happen to the file if you wrote at the record position of account number 150?
3. Write a function `deleteAccount` that removes a customer from the bank file by writing a blank record over it. Why does this need `"rb+"` and not `"wb"`?
4. Fill in a customer with account number 100, and run the `feof` version of the reading loop with and without the `result != 0` test. Explain exactly what happens on the last trip through the loop in each case.
5. Print `sizeof(struct clientData)` and the sum of `sizeof` of its four members. Explain the difference. Then try a structure `struct { char a; int b; char c; }` and the same structure with the members in the order `int b; char a; char c;`. What sizes do you get, and why do they differ?